



Implementing Client Script & UI Policy

Incident Management Automation in ServiceNow

ServiceNow Project Documentation | Phase-wise Delivery

Module: Incident [incident] | Client Script: Dynamic incident behavior | UI Policy:
Conditional field control

Project Phases Overview

01

Ideation

Problem definition & user experience

02

Requirement Analysis

Functional & non-functional needs

03

Project Design

Client Script + UI Policy architecture

04

Project Planning

Timeline, roles & validation plan

05

Development

Configure, script, test & debug

06

Documentation

Test evidence & handover

1. Ideation Phase



Problem Statement

Manual incident handling can lead to inconsistent field values, missing resolution information, and unnecessary user effort. Agents may need to manually enforce conditions such as making resolution fields mandatory when an incident is resolved.

Improve Incident Data Quality

Use client-side automation to guide agents, reduce incomplete records, and make incident form behavior consistent.

Technical Opportunity

Combine an Incident Client Script with a UI Policy so the form can react to field changes and conditionally make fields mandatory, visible, or read-only.

User Impact

Agents receive immediate guidance on the form, while support teams get cleaner incident records and more consistent closure information.

2. Requirement Analysis

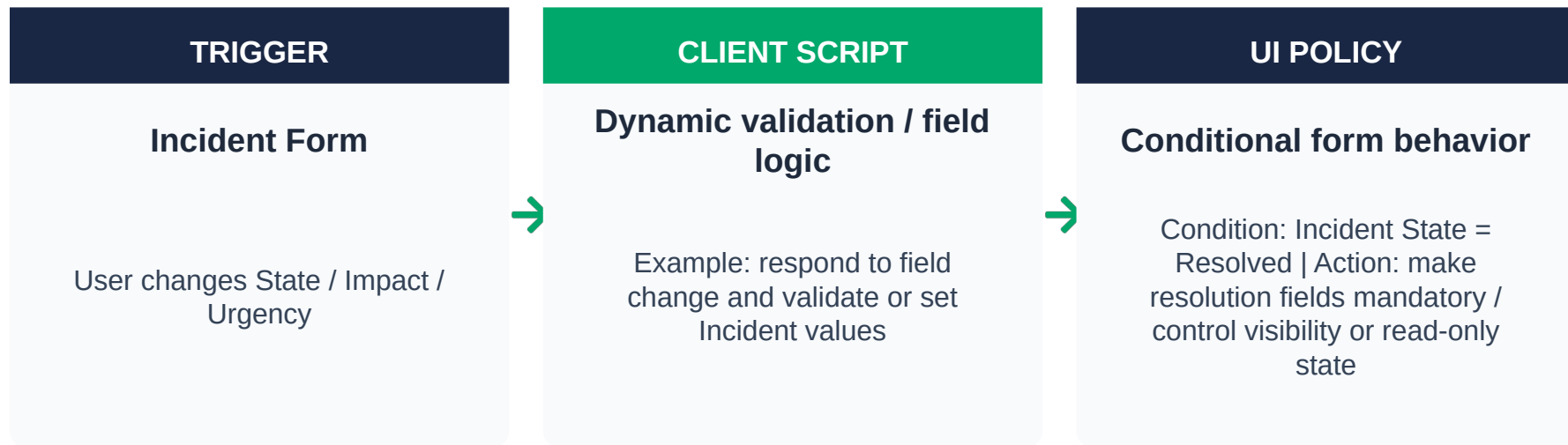
Functional Requirements

- ▶ Configure an Incident Client Script for dynamic form behavior
- ▶ Trigger logic on relevant Incident field changes
- ▶ Validate or derive field values on the client side
- ▶ Configure UI Policy conditions for Incident records
- ▶ Make Resolution notes / Close notes mandatory when appropriate
- ▶ Control field visibility or read-only state based on incident state
- ▶ Prevent unnecessary scripting where a UI Policy can handle the requirement

Non-Functional Requirements

- ▶ Client-side execution only; no unnecessary server calls
- ▶ Clear, readable and reusable JavaScript
- ▶ Compatible with the Incident form and existing configurations
- ▶ Avoid conflicts with other Client Scripts / UI Policies
- ▶ Immediate response when users change relevant fields
- ▶ Tested for new, existing and updated incidents
- ▶ Maintainable configuration with clear naming and documentation

3. Project Design – Client Script & UI Policy Architecture



Key Design Decisions

- Use UI Policy for simple field-state control. |
- Use Client Script for dynamic calculations or validation. |
- Keep logic client-side and avoid duplicate rules. |
- Test execution order and interaction with existing Incident configurations.

3. Project Design – Data & Field Mapping

Field / Element	Condition / Source	Form Action
State	Incident.state = Resolved	Trigger UI Policy condition
Resolution Notes	Mandatory when State = Resolved	Set mandatory = true
Close Notes	Mandatory when State = Resolved	Set mandatory = true
Close Code	Mandatory when State = Resolved	Set mandatory = true
Impact / Urgency	Values selected on Incident form	Client Script can validate / calculate related values
Short Description	User-entered Incident value	Client Script can validate required content
Form Messages	Client-side validation result	Show user-friendly message when validation fails

4. Project Planning

High-Level Timeline



Key Roles & Responsibilities

Process Owner	Define incident process rules and success criteria
ServiceNow Developer	Create Client Script and UI Policy configurations
Incident Admin	Validate form behavior and field requirements
Tester / UAT	Execute positive, negative and regression test cases
Documentation Owner	Record configuration, test evidence and handover notes

5. Project Development – Implementation Steps

- 1 Create Client Script**

Incident → Client Scripts → New → choose the appropriate Type (for example, onChange) and field.
- 2 Define Trigger**

Select the Incident field that should trigger the logic, such as State, Impact or Urgency.
- 3 Add Script Logic**

Use g_form methods to validate, read or set client-side Incident fields; keep the logic focused and readable.
- 4 Create UI Policy**

Incident → UI Policies → New → define condition such as State is Resolved.
- 5 Configure UI Policy Actions**

Set Resolution Notes, Close Notes and Close Code to mandatory; optionally control visibility or read-only behavior.
- 6 Test & Activate**

Save and test with new and existing incidents; confirm the form reacts correctly and does not conflict with other configurations.

5. Development – Test Evidence

Requested Item

Scenario 1 – Active Incident

State: In Progress

Resolution fields are not forced unnecessarily.

Client Script executes only for its configured trigger.

No unwanted field changes occur.

- Variables: Optional Software available

Scenario 2 – Resolve Incident

State changed to Resolved

Resolution Notes becomes mandatory.

Close Notes becomes mandatory.

Close Code becomes mandatory.

UI Policy actions are applied immediately.

- Assignment group: Hardware

Scenario 3 – Validation

Invalid / incomplete input entered

Client Script displays a clear validation message.

Record submission is blocked when the configured rule fails.

Valid input allows the user to continue.

Regression check

Existing Incident fields and standard form behavior remain usable.

6. Project Documentation & Handover



Flow Design Spec

Configuration Spec

- Client Script name, type and table
- Trigger field and JavaScript purpose
- UI Policy condition and UI Policy Actions



Test Cases

- Active → Resolved transition
- Mandatory field validation
- Client Script positive / negative cases
- Regression and conflict checks



Knowledge Transfer

- Where to edit Client Scripts and UI Policies
- How to activate / deactivate safely
- How to troubleshoot client-side behavior
- Naming and maintenance guidance



Operational Guide

- Check browser console / form messages when debugging
- Review conflicting Client Scripts and UI Policies
- Validate changes in a non-production instance first
- Follow change control before deployment

Success Metrics & Next Steps

100%

Required closure fields |
enforced by policy

Immediate

Client-side | form
response

0

Unnecessary | manual
prompts

Consistent

Incident | data quality

Recommended Next Steps

1. Extend UI Policies to additional Incident conditions where appropriate.
2. Add targeted Client Scripts for advanced validation or dynamic calculations.
3. Review existing Incident Client Scripts / UI Policies for conflicts and duplicate logic.
4. Add UAT screenshots and execution results to the project evidence.
5. Document the final configuration in the ServiceNow knowledge base.



Project Complete

Incident Client Script & UI Policy configuration is ready for controlled deployment.

Module: Incident • Client Script: Dynamic client-side logic • UI Policy: Conditional field behavior | Result: Incident forms provide consistent, guided data entry and closure validation.

ServiceNow | Incident Management | Client Script | UI Policy